

# UNIVERSIDAD AUTONOMA DE OCCIDENTE

UAO Virtual

*Especializacion en Inteligencia Artificial*

---

## INFORME DE PRACTICA

### Guia M2A3 — Balanceo de Carga con HAProxy y LXD

---

|                    |                                                                  |
|--------------------|------------------------------------------------------------------|
| <b>Estudiante:</b> | Milton Jaramillo                                                 |
| <b>Docente:</b>    | Prof. Oscar Mondragon                                            |
| <b>Materia:</b>    | Computacion en la Nube                                           |
| <b>Modulo:</b>     | 2 — Introduccion a la virtualizacion de recursos computacionales |
| <b>Practica:</b>   | Guia M2A3 — Balanceo de Carga con HAProxy y LXD                  |
| <b>Año:</b>        | 2026                                                             |

# 1. Introduccion

---

El balanceo de carga es una tecnica fundamental en arquitecturas de alta disponibilidad que distribuye el trafico entrante entre multiples servidores, evitando que un unico nodo concentre toda la carga y se convierta en un punto de fallo. En entornos de computacion en la nube, esta capacidad es esencial para garantizar la continuidad del servicio y la escalabilidad horizontal de las aplicaciones.

HAProxy (High Availability Proxy) es uno de los balanceadores de carga de codigo abierto mas ampliamente utilizados en entornos de produccion. Soporta multiples algoritmos de distribucion de trafico como Round Robin, Least Connections y Source IP Hashing, y ofrece un panel de estadisticas en tiempo real que permite monitorear el estado de los servidores del backend.

Esta practica integra dos tecnologias vistas en el modulo: los contenedores LXD como plataforma de despliegue ligera, y HAProxy como capa de balanceo de carga. Se reutiliza la infraestructura Vagrant del Modulo 1 (clienteUbuntu y servidorUbuntu) y sobre ella se crean tres contenedores LXD: dos servidores web Apache (web1 y web2) y un contenedor balanceador (haproxy). La practica incluye pruebas de tolerancia a fallos y pruebas de carga con Apache JMeter para evaluar el comportamiento del sistema bajo condiciones de estres.

## 2. Objetivos

---

### 2.1 Objetivo General

Implementar un sistema de balanceo de carga con HAProxy sobre contenedores LXD en un entorno Vagrant, evaluando su comportamiento ante fallos de servidores y bajo carga concurrente con Apache JMeter.

### 2.2 Objetivos Especificos

- Crear y configurar tres contenedores LXD (web1, web2 y haproxy) sobre la VM servidorUbuntu del entorno Vagrant existente.
- Instalar y configurar Apache2 en los contenedores web1 y web2 con paginas personalizadas que permitan identificar a que servidor responde el balanceador.
- Configurar HAProxy con algoritmo Round Robin para distribuir el trafico HTTP entre los dos servidores web.
- Habilitar el dashboard de estadisticas de HAProxy para monitorear el estado de los nodos del backend en tiempo real.
- Realizar pruebas de tolerancia a fallos apagando los contenedores web1 y web2 individualmente y en conjunto, verificando el comportamiento del balanceador.
- Ejecutar pruebas de carga con Apache JMeter con 200 usuarios concurrentes y 10,000 peticiones totales para medir el rendimiento del sistema.

## 3. Herramientas y Tecnologias Utilizadas

---

Para el desarrollo de esta practica se utilizaron las siguientes herramientas y tecnologias:

- VirtualBox 7.2.8: Hipervisor de tipo 2 que provee la infraestructura de virtualizacion base para las maquinas virtuales Vagrant.
- Vagrant 2.4.9: Herramienta de automatizacion de entornos que gestiona las VMs clienteUbuntu (192.168.100.2) y servidorUbuntu (192.168.100.3) con Ubuntu 20.04 LTS.
- LXD sobre Ubuntu 20.04: Gestor de contenedores del sistema usado para crear y administrar los contenedores web1, web2 y haproxy sobre la VM servidorUbuntu.
- Apache2: Servidor web instalado en los contenedores web1 (10.149.16.40) y web2 (10.149.16.101) con paginas HTML personalizadas para identificar cada nodo.
- HAProxy 2.0.33: Balanceador de carga instalado en el contenedor haproxy (10.149.16.127), configurado con algoritmo Round Robin y panel de estadisticas habilitado en /haproxy?stats.
- Apache JMeter 5.6.3: Herramienta de pruebas de carga utilizada para simular 200 usuarios concurrentes con ramp-up de 1 segundo y 50 iteraciones, generando 10,000 peticiones HTTP totales al balanceador.

## 4. Descripcion del Proceso

---

### 4.1 Creacion de Contenedores LXD

Partiendo de la infraestructura Vagrant del Modulo 1, se accedio a la VM servidorUbuntu mediante `vagrant ssh servidorUbuntu`. Se crearon los tres contenedores en un unico comando encadenado: `lxc launch ubuntu:20.04 web1 && lxc launch ubuntu:20.04 web2 && lxc launch ubuntu:20.04 haproxy`. Una vez iniciados, se verifico su estado con `lxc list`, confirmando que los tres contenedores se encontraban en estado RUNNING con sus respectivas IPs: web1 en 10.149.16.40, web2 en 10.149.16.101 y haproxy en 10.149.16.127.

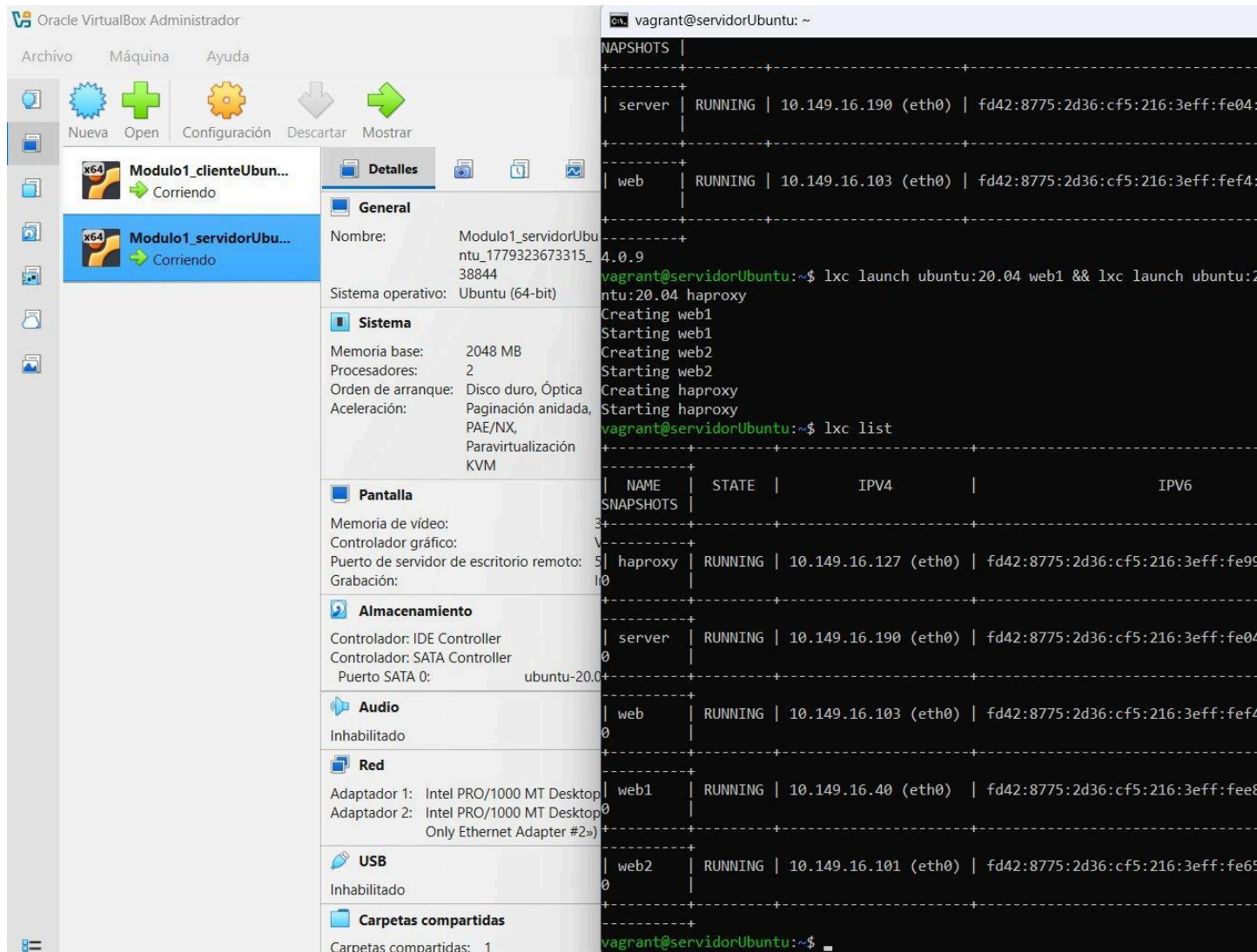


Figura 1. Contenedores web1, web2 y haproxy en estado RUNNING con sus IPs asignadas

## 4.2 Configuración de Servidores Web Apache

Se instaló Apache2 en los contenedores web1 y web2 utilizando lxc exec. En cada contenedor se actualizó el repositorio, se instaló apache2 y se sobrescribió el archivo /var/www/html/index.html con el contenido identificador de cada nodo: 'Hello from web1' y 'Hello from web2' respectivamente. Se verificó el correcto funcionamiento de ambos servidores desde el host con curl 10.149.16.40 y curl 10.149.16.101, obteniendo las respuestas esperadas de cada contenedor.

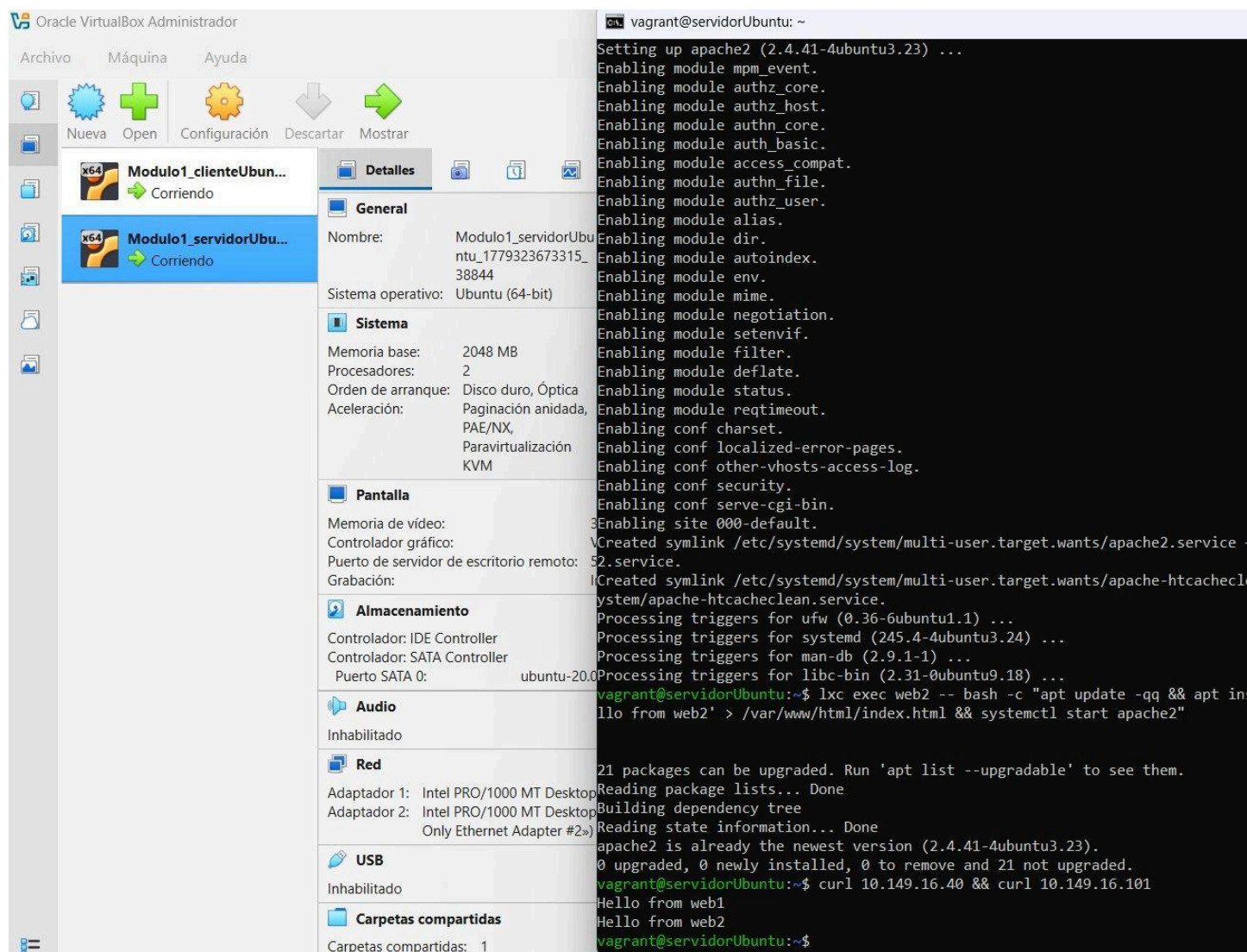


Figura 2. Respuestas de Apache en web1 y web2 verificadas con curl desde servidorUbuntu

### 4.3 Configuración de HAProxy con Round Robin

Dentro del contenedor haproxy se instaló el paquete haproxy. Se editó el archivo `/etc/haproxy/haproxy.cfg` para definir el frontend HTTP escuchando en el puerto 80 y el backend web-backend con algoritmo roundrobin, apuntando a las IPs internas de web1 y web2. Se habilitó también el panel de estadísticas con usuario admin y contraseña admin en la ruta `/haproxy?stats`. Tras reiniciar el servicio, se configuró el reenvío de puertos LXD con `lxc config device add haproxy http proxy listen=tcp:0.0.0.0:80 connect=tcp:10.149.16.127:80` para exponer el balanceador en la IP de servidorUbuntu (192.168.100.3).

### 4.4 Verificación del Balanceo en Navegador

Se comprobó el funcionamiento del balanceo accediendo desde el navegador del clienteUbuntu a la dirección 192.168.100.3. Recargando la página de forma consecutiva, el balanceador fue distribuyendo las peticiones alternadamente entre los dos nodos, mostrando primero 'Hello from

web1' y en la siguiente solicitud 'Hello from web2', confirmando el correcto funcionamiento del algoritmo Round Robin.

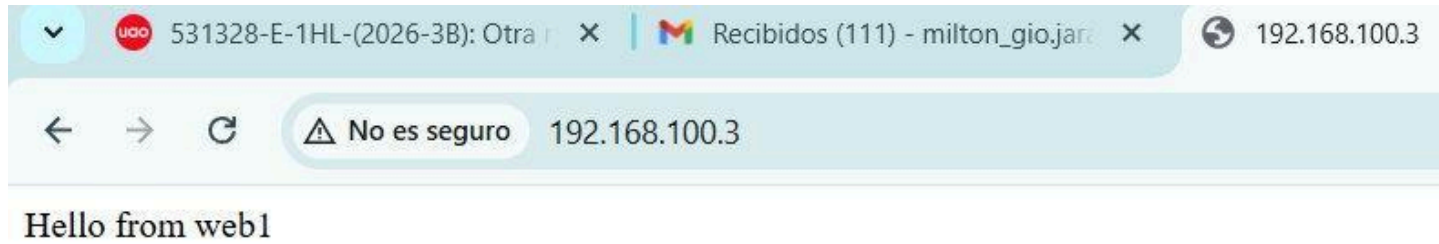


Figura 3. Navegador mostrando respuesta de web1 a traves del balanceador HAProxy

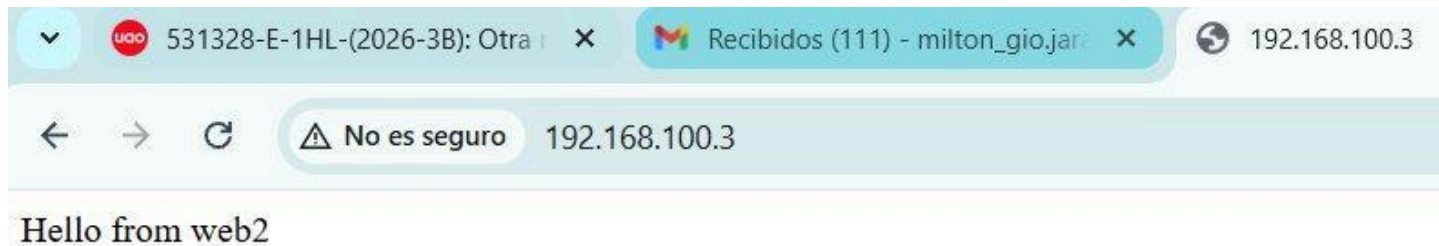


Figura 4. Navegador mostrando respuesta de web2 a traves del balanceador HAProxy

## 4.5 Dashboard de Estadísticas HAProxy



Se accedio al panel de estadisticas de HAProxy en <http://192.168.100.3/haproxy?stats> con las credenciales admin/admin. El dashboard mostro en tiempo real el estado del frontend HTTP y del backend web-backend, con web1 y web2 en color verde (estado UP), el numero de sesiones activas, bytes transferidos, tiempo de respuesta del health check (L4OK in 0ms) y el conteo total de peticiones procesadas por cada nodo.

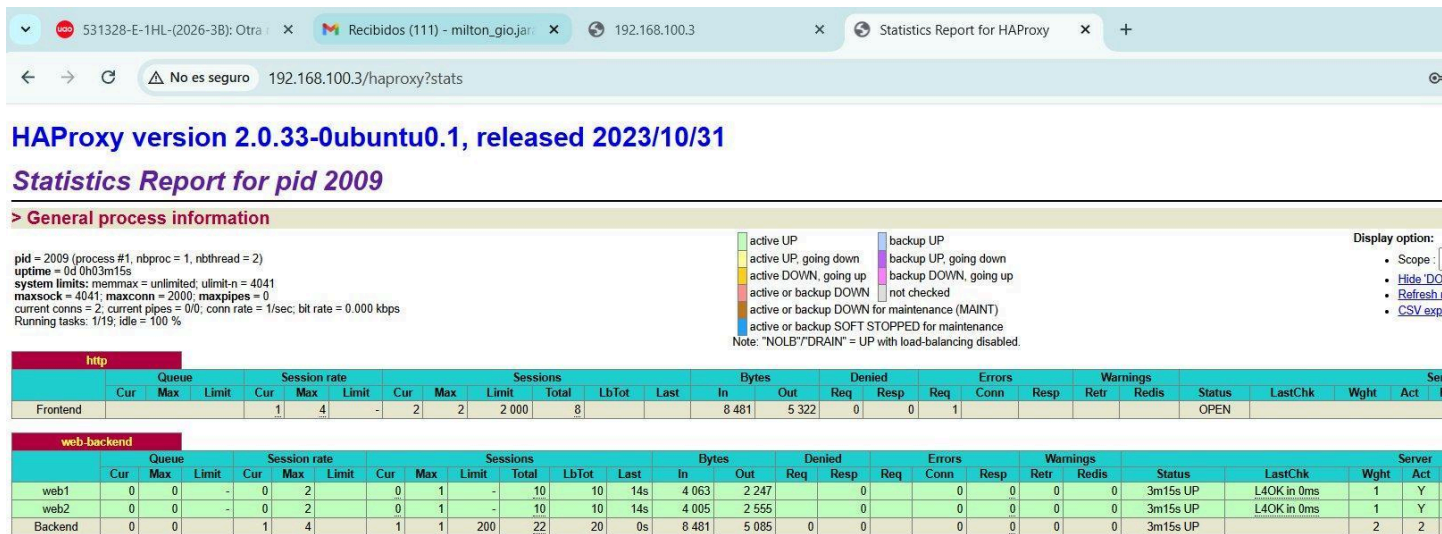


Figura 5. Dashboard de HAProxy con web1 y web2 en estado UP (verde)

#### 4.6 Pruebas de Tolerancia a Fallos

Se realizaron dos pruebas de fallo para verificar la resiliencia del sistema. En la primera prueba se apago el contenedor web2 con `lxc stop web2`. El dashboard de HAProxy detecto automaticamente la caida del nodo en el siguiente health check y lo marco en rojo (estado DOWN), redirigiendo el 100% del trafico hacia web1 sin interrupcion del servicio para el usuario final.

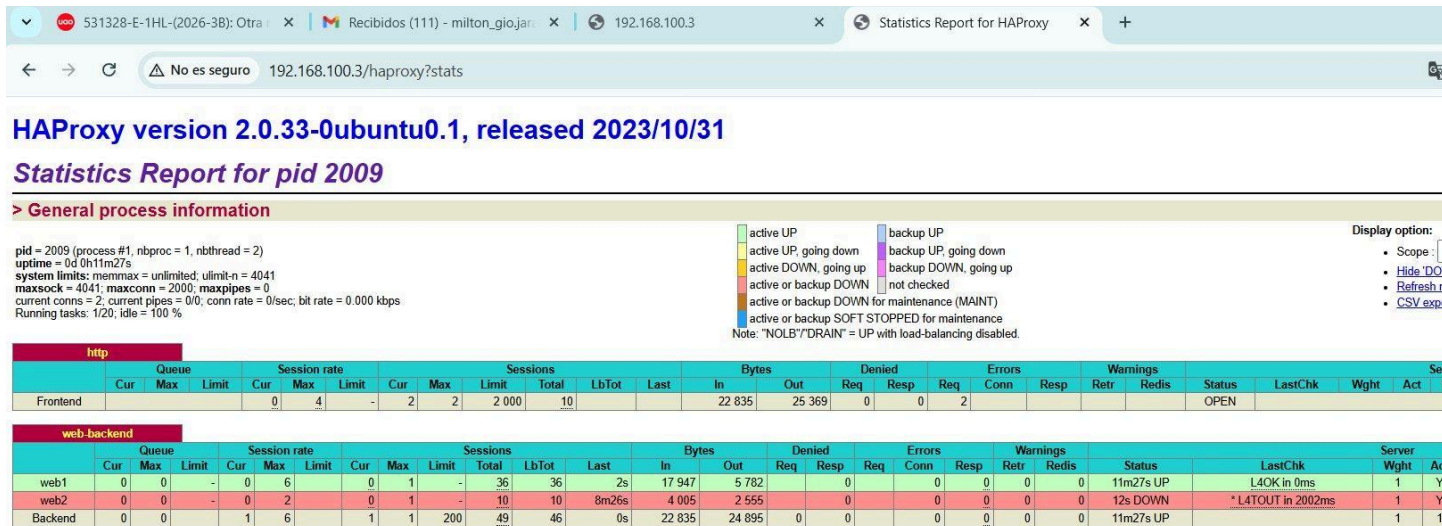


Figura 6. Dashboard de HAProxy con web2 en estado DOWN y todo el trafico dirigido a web1

En la segunda prueba se apago tambien web1, dejando ambos servidores del backend sin disponibilidad. En este escenario, HAProxy no tenia ningun nodo activo al cual redirigir las peticiones y devolvio un error HTTP 503 Service Unavailable, indicando que ningun servidor estaba disponible para atender la solicitud.

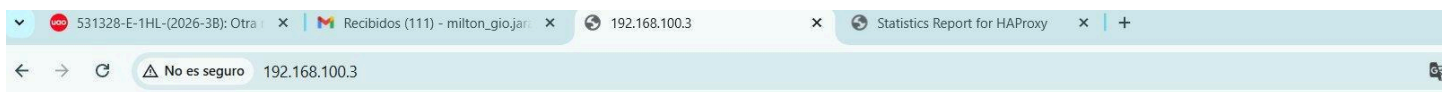


Figura 7. Error 503 Service Unavailable al estar ambos servidores web caidos

## 4.7 Pruebas de Carga con Apache JMeter



Con ambos contenedores web1 y web2 nuevamente activos, se ejecutaron pruebas de carga desde el cliente Ubuntu utilizando Apache JMeter 5.6.3. Se configuro un Thread Group con 200 usuarios virtuales, ramp-up de 1 segundo y 50 iteraciones por usuario, generando un total de 10,000 peticiones HTTP al balanceador en 192.168.100.3. El listener View Results Tree mostro todas las peticiones con icono verde, confirmando que el balanceador proceso exitosamente la carga distribuida entre ambos nodos sin errores de conectividad.

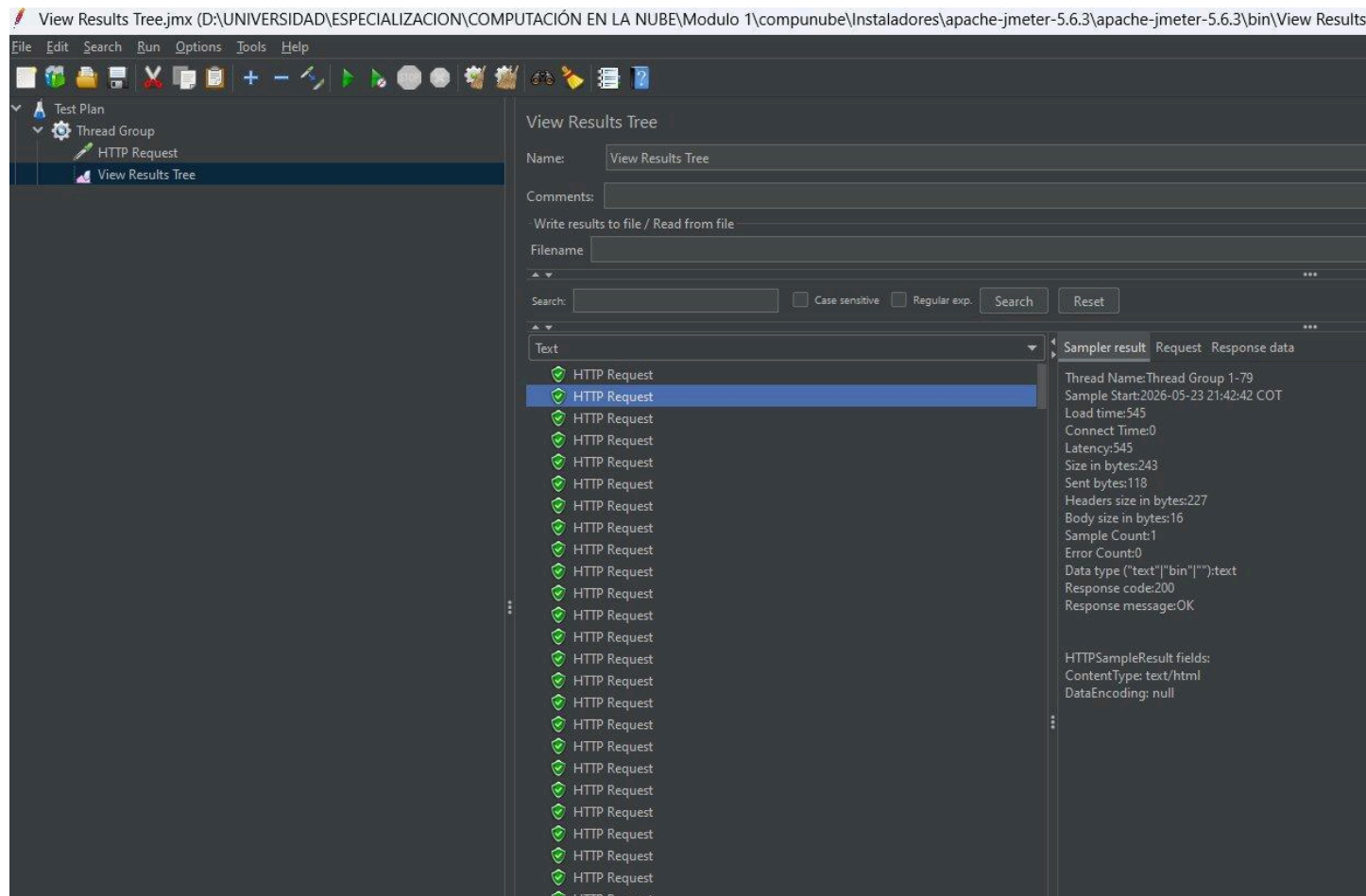


Figura 8. JMeter View Results Tree mostrando todas las peticiones HTTP exitosas (codigo 200)

El Summary Report de JMeter arrojo los siguientes resultados para las 10,000 peticiones: throughput de 379.1 solicitudes por segundo, tiempo de respuesta promedio de 358ms, tiempo minimo de 7ms, tiempo maximo de 2203ms, desviacion estandar de 215.11ms y tasa de error del 0.45%. Estos resultados confirman que el sistema de balanceo distribuyo eficientemente la carga y mantuvo una alta disponibilidad durante la prueba.

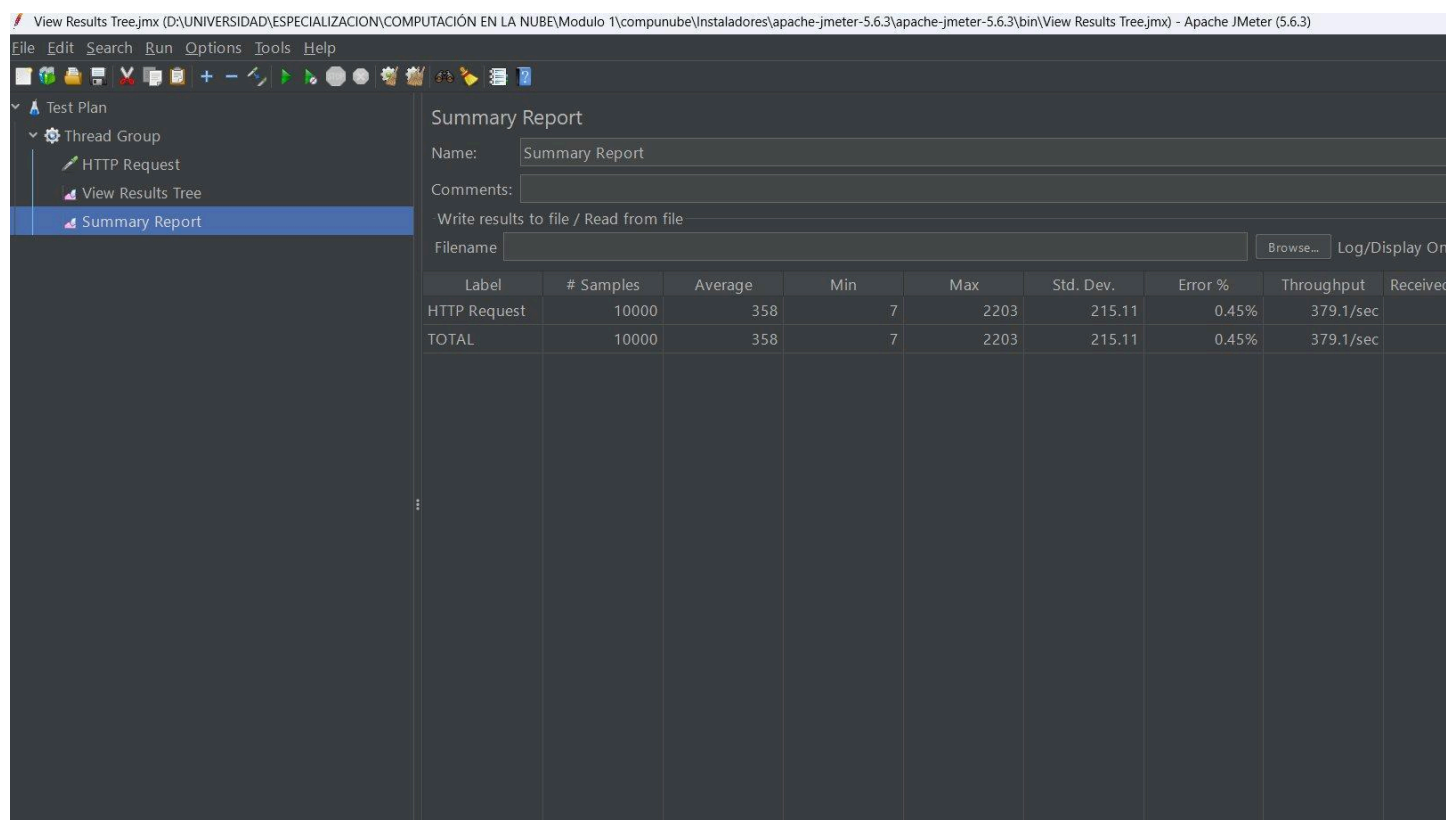


Figura 9. JMeter Summary Report con metricas de rendimiento de las 10,000 peticiones

## 5. Conclusiones

La implementacion de HAProxy como balanceador de carga sobre contenedores LXD demostro ser una solucion eficiente y de bajo costo de recursos para distribuir trafico HTTP en entornos de desarrollo y pruebas. La combinacion de ambas tecnologias permite replicar arquitecturas de produccion reales dentro de una unica maquina virtual, reduciendo significativamente los requisitos de hardware.

El algoritmo Round Robin de HAProxy distribuyo equitativamente las 10,000 peticiones de la prueba JMeter entre web1 y web2, logrando un throughput de 379.1 req/seg con solo un 0.45% de errores y un tiempo de respuesta promedio de 358ms. Estos resultados evidencian la capacidad del sistema para manejar carga concurrente alta con una latencia aceptable.

Las pruebas de tolerancia a fallos confirmaron el comportamiento esperado del balanceador: ante la caida de un nodo, HAProxy detecto la indisponibilidad mediante health checks y redirigió automaticamente el trafico al nodo activo sin interrupcion perceptible para el usuario. Este mecanismo de failover automatico es fundamental en arquitecturas de alta disponibilidad en la nube.

El panel de estadisticas de HAProxy resulto ser una herramienta valiosa para el monitoreo en tiempo real del sistema, permitiendo observar el estado de cada nodo, las sesiones activas, los

bytes transferidos y los tiempos de respuesta de los health checks sin necesidad de herramientas externas adicionales.

En conclusion, esta practica refuerza los conceptos de escalabilidad horizontal y alta disponibilidad aplicados en entornos cloud, sentando las bases para comprender soluciones mas avanzadas como NGINX Plus, AWS Elastic Load Balancer, Google Cloud Load Balancing o Azure Load Balancer utilizadas en produccion a escala empresarial.

## 6. Referencias

---

HAProxy Technologies. (2026). HAProxy — The Reliable, High Performance TCP/HTTP Load Balancer. Recuperado de <http://www.haproxy.org/>

Linux Containers. (2026). LXD Introduction. Recuperado de <https://linuxcontainers.org/>

Apache Software Foundation. (2026). Apache JMeter. Recuperado de <https://jmeter.apache.org/>

Mondragon, O. (2026). Guia M2A3 - Balanceo de Carga con HAProxy y LXD. UAO Virtual, Computacion en la Nube, Modulo 2.

Jaramillo, M. (2026). Repositorio compunube. GitHub. Recuperado de <https://github.com/milton329/compunube>